

Práctico 2: Modelado y diseño de bases NoSQL (MongoDB)

Objetivo:

Diseñar modelos de datos en MongoDB aplicando los patrones embebido, referenciado o híbrido, analizando el impacto de cada decisión según el contexto de uso.

Formato de entrega

- Documento PDF con las respuestas teóricas
- Capturas de pantalla de los comandos ejecutados en mongosh
- Código de los comandos en formato texto (para que pueda copiarse)

ACTIVIDAD 1: Plataforma Educativa - Patrón Híbrido

- 1) Una plataforma educativa almacena Cursos, Estudiantes, y Evaluaciones. Cada curso tiene muchos estudiantes, y cada estudiante puede realizar varias evaluaciones dentro de un curso.
 - a) Diseñar el documento de Curso aplicando un patrón híbrido, donde se embeban los datos esenciales de las evaluaciones (promedio y cantidad), pero las evaluaciones completas se almacenen en otra colección llamada evaluaciones.
Incluir en tu respuesta:
 - Estructura JSON del documento Curso
 - Estructura JSON de la colección evaluaciones
 - Diagrama o esquema que muestre la relación
 - b) Explicar brevemente por qué el patrón híbrido es más adecuado que usar solo embebido o solo referencia en este caso.
- 2) Conectarse a mongosh y crear la base de datos 'plataforma_educativa'. Implementar el diseño propuesto.
 - a) Crear la colección cursos e insertar 2 cursos con datos embebidos de resumen de evaluaciones.
 - b) Crear la colección 'evaluaciones' con las evaluaciones detalladas que referencian a los cursos.
 - c) Realizar una consulta que obtenga un curso específico con su resumen de evaluaciones.
 - d) Realizar una consulta que obtenga todas las evaluaciones detalladas de un curso específico.
 - e) Realizar una agregación que calcule el promedio de notas por curso.

ACTIVIDAD 2 : Relacion estudiante-tutor - Muchos a Muchos

- 1) Cada estudiante puede tener uno o más tutores, y un tutor puede acompañar a varios estudiantes. Para cada tutoría se guarda la fecha de inicio y el tipo de acompañamiento (académico, motivacional o técnico).
 - a) Identificar qué tipo de relación se forma entre Estudiante y Tutor (uno-a-uno, uno-amuchos, muchos-a-muchos).
 - b) Indicar qué patrón conviene usar: embebido, referencia o colección asociativa. Justificar tu respuesta.
 - c) Diseñar la estructura de documentos que represente esta relación, incluyendo los campos 'fecha_inicio' y 'tipo_acompanamiento'.
- 2) Implementar el diseño propuesto usando el patrón de colección asociativa.
 - a) Crear la colección 'estudiantes' con al menos 3 estudiantes.
 - b) Crear la colección 'tutores' con al menos 2 tutores.
 - c) Crear la colección asociativa 'tutorias' que relacione estudiantes con tutores.
 - d) Consultar todos los estudiantes de un tutor específico.

ACTIVIDAD 3 : Biblioteca digital - Recursos compartidos

- 1) Una biblioteca digital almacena Materias y Recursos (archivos PDF, videos o enlaces). Cada materia tiene muchos recursos, pero los recursos pueden ser reutilizados en distintas materias.
 - a) ¿Qué patrón de modelado usarías para vincular Materias y Recursos? (embebido, referencia o híbrido)
 - b) Explicar brevemente por qué el embebido NO sería adecuado en este caso.
 - c) Diseñar un ejemplo de documento Materia y uno de Recurso, indicando cómo se relacionan.
- 2) Implementación práctica.
 - a) Crear la colección 'recursos' con recursos que puedan ser compartidos.
 - b) Crear la colección 'materias' que referencie a los recursos.
 - c) Consultar en cuántas materias se usa un recurso específico.
 - d) Listar todas las materias que usan un recurso específico.
 - e) Agregar un nuevo recurso a una materia existente.

1) Diseño de Modelo y Justificación

Estructura JSON (Curso):

```
{
  "_id": ObjectId("..."),
  "nombre": "Programación 3",
  "estudiantes_count": 45,
  "resumen_evaluaciones": {
    "promedio_general": 8.5,
    "total_evaluaciones": 120
  }
}
```

Estructura JSON (Evaluaciones):

```
{
  "_id": ObjectId("..."),
  "curso_id": ObjectId("..."),
  "estudiante": "Damian Tristant",
  "nota": 9,
  "fecha": "2026-04-26"
}
```

Esquema de Relación: Se utiliza una relación 1:N (Uno a Muchos). El curso guarda un resumen (embebido) para acceso rápido, mientras que el detalle vive en una colección separada (referenciado) para evitar que el documento del curso crezca indefinidamente.

Relacion:

[Colección: cursos]

```
{
  _id: ObjectId,
  nombre: String,
  resumen_evaluaciones: { <--- (Embebido)
    promedio: Number,
    cantidad: Number
  }
}
```

|
| (Referencia por ID)

v

[Colección: evaluaciones]

```
{
  _id: ObjectId,
  curso_id: ObjectId, <--- (Foreign Key)
  estudiante: String,
}
```

```
    nota: Number
  }
```

Justificación del Patrón Híbrido: Es ideal porque permite consultar los datos que más se usan (promedio y cantidad) sin salir del documento del curso (eficiencia en lectura), pero mantiene el historial completo de evaluaciones por separado para no saturar la memoria ni exceder el límite de 16MB de MongoDB.

2) Implementación en mongosh

```
// Crear y usar la base de datos
use plataforma_educativa
```

```
// a) Insertar cursos con resumen embebido
db.cursos.insertMany([
  { nombre: "Base de Datos NoSQL", resumen_evaluaciones: { promedio: 7.5, cantidad: 10 } },
  { nombre: "Desarrollo Web", resumen_evaluaciones: { promedio: 8.2, cantidad: 15 } }
]);
```

```
// b) Insertar evaluaciones detalladas
db.evaluaciones.insertMany([
  { curso_id: db.cursos.findOne({ nombre: "Base de Datos NoSQL" })._id, nota: 8, estudiante: "Estudiante A" },
  { curso_id: db.cursos.findOne({ nombre: "Base de Datos NoSQL" })._id, nota: 7, estudiante: "Estudiante B" }
]);
```

```
// c) Consulta de curso con su resumen
db.cursos.find({ nombre: "Base de Datos NoSQL" });
```

```
// d) Consultar evaluaciones de un curso
var cursoId = db.cursos.findOne({ nombre: "Base de Datos NoSQL" })._id;
db.evaluaciones.find({ curso_id: cursoId });
```

```
// e) Agregación para promedio
db.evaluaciones.aggregate([
  { $group: { _id: "$curso_id", promedio_notas: { $avg: "$nota" } } }
]);
```

Actividad 2: Relación Estudiante-Tutor

1) Análisis de Relación Tipo de relación: Muchos-a-Muchos (N:M). Un estudiante tiene varios tutores y un tutor tiene varios estudiantes. Patrón sugerido: Colección Asociativa. Justificación: Al ser una relación N:M con datos adicionales (fecha y tipo de acompañamiento), una colección intermedia permite gestionar estos atributos sin duplicar datos innecesariamente en las colecciones principales de Estudiantes o Tutores.

2) Implementación JavaScript// a) Colección estudiantes

```
db.estudiantes.insertMany([ { nombre: "Ana" }, { nombre: "Luis" }, { nombre: "Marta" } ]);
```

// b) Colección tutores

```
db.tutores.insertMany([ { nombre: "Tutor Tecnico" }, { nombre: "Tutor Academico" } ]);
```

// c) Colección asociativa tutorias

```
db.tutorias.insertOne({
  estudiante_id: db.estudiantes.findOne({ nombre: "Ana" })._id,
  tutor_id: db.tutores.findOne({ nombre: "Tutor Tecnico" })._id,
  fecha_inicio: new Date(),
  tipo_acompanamiento: "técnico"
});
```

// d) Consultar estudiantes de un tutor

```
var tutorId = db.tutores.findOne({ nombre: "Tutor Tecnico" })._id;
db.tutorias.find({ tutor_id: tutorId }, { estudiante_id: 1 });
```

Actividad 3: Biblioteca Digital

1) Modelado de Recursos

Patrón: Referencia.

¿Por qué NO embebido?: Si embebemos los recursos en cada materia, cuando un recurso se actualice (ej. corregir un PDF), tendríamos que buscarlo y actualizarlo en todas las materias donde aparezca. Esto genera inconsistencia de datos y redundancia.

Diseño: La colección materias tendrá un array de IDs que apuntan a la colección recursos

2) Implementación Práctica

// a) Crear recursos

```
db.recursos.insertMany([
  { _id: 1, titulo: "Manual MongoDB", tipo: "PDF" },
  { _id: 2, titulo: "Video Tutorial NoSQL", tipo: "Video" }
]);
```

// b) Crear materias con referencias

```
db.materias.insertMany([
  { nombre: "Sistemas", recursos_ids: [1, 2] },
  { nombre: "Programación", recursos_ids: [1] }
]);
```

// c) Contar materias que usan un recurso (ID: 1)

```
db.materias.countDocuments({ recursos_ids: 1 });
```

// d) Listar materias que usan el recurso 1

```
db.materias.find({ recursos_ids: 1 }, { nombre: 1 });
```

// e) Agregar nuevo recurso a una materia

```
db.materias.updateOne(
  { nombre: "Programación" },
  { $push: { recursos_ids: 2 } }
);
```